User Manual

1. Installation

Install a C++17 compiler, CMake 3.20 or newer, and the system libraries needed by raylib. On Debian/Ubuntu:

```
sudo apt install build-essential cmake libgl1-mesa-dev libx11-dev \
    libxrandr-dev libxinerama-dev libxcursor-dev libxi-dev
```

CUDA is optional. If `nvcc` is available, CMake can build optional GPU tensor kernels; otherwise the CPU implementation is used.

2. Running the program

Build and run from the repository root:

```
./scripts/build.sh
./scripts/run.sh
```

The default submitted configuration starts in a fullscreen window with the activation panel set to OFF. The `window.width` and `window.height` values are used as the windowed fallback if fullscreen is disabled. The app reads `config.ini` at startup for window, font, model, default FEN, and clock settings.

Before a model demo, populate local weight files:

```
./scripts/import_models.sh
```

The full visual tutorial is available as `tutorial.gif`, with a higher-resolution video copy at `demo.mp4`. It walks through the main command buttons, architecture selectors, abstract/detailed activation views, setup editor, FEN tools, and native search preview.

Before the final defense, run:

```
./scripts/demo_check.sh
```

This verifies the required documents, tutorial media, local model files, fullscreen/OFF startup configuration, media metadata, and automated tests.

3. Playing a game

The app currently supports human-vs-human chess play. Move input is mouse based: click a piece, then click a legal destination. The selected piece and legal targets are highlighted on the board. Dragging a piece to a legal destination is also supported.

After each legal move, the status banner, NN activation panel, and move history update. The move list shows SAN in two columns and can be navigated with the mouse or keyboard history shortcuts.

Use Undo or `Ctrl+Z` to step backward through the move history, and Redo or `Ctrl+Y` to step forward again. Rewinding to the start restores the board to the initial position while keeping the move history visible.

Pawn promotion opens a modal picker. Choose the promotion piece with the mouse; press `Esc` to cancel the pending selection.

4. Command buttons

The top-level control grid has these commands:

Button	Action
Reset	Restores the configured start position and clears transient UI state.
Flip	Reverses board orientation without changing the position.
Undo	Steps one ply backward through the move-history linked list.
Redo	Steps one ply forward after an undo.
Random	Plays one random legal move from the current position.
Search	Starts or stops the native engine-style search preview for the selected architecture.
Load FEN	Opens the FEN input dialog.
Save FEN	Writes the current position to `position.fen`.
Setup	Opens the board editor.
Edit FEN	Opens the current position as editable FEN text.

5. Loading and saving positions (FEN)

Click Load FEN or press `L` to open the FEN dialog. Enter a complete six-field FEN string and press Enter. Invalid input stays in the dialog and shows an inline error.

Click Save FEN or press `S` to write the current position to `position.fen`. The save path is fixed for now.

6. Board editor

Click Setup or press `A` to enter editor mode. Select a piece from the palette, left-click a square to place it, or right-click a square to erase it. The palette also has Eraser, Clear, and Startpos buttons.

Use the editor panel to set side to move, castling rights, en-passant target, halfmove clock, and fullmove number. Click Validate before Apply. Apply is enabled only when the edited position passes validation.

Invalid positions stay in editor mode and show the first validation issues.

7. Switching architectures

The right panel has architecture buttons for NNUE, CNN, BT4, and OFF. Selecting an architecture changes both the summary label and the activation visualization. Each architecture view has an abstract/detailed toggle:

- NNUE shows active feature counts, accumulator/clipped bars, and a simple feature-to-accumulator-to-value diagram.
- CNN shows board-plane heatmaps, policy/value/WDL summaries, and a residual block diagram when compatible weights are loaded.
- BT4 shows the 64-token transformer pipeline, synthetic token/attention grids, deterministic attention snapshots, policy logits, and WDL/value summaries from the native C++ BT4-style path.

8. Reading the activation views

8.1 NNUE view

NNUE view highlights HalfKP-style feature activity, accumulator ranges, clipped post-ReLU activity, and the current scalar evaluation.

8.2 CNN view

CNN view summarizes the encoded board planes, residual stack activity, policy head, and WDL/value head. If weights are missing, the panel still shows the expected pipeline and reports the load status.

8.3 BT4 view

BT4 view shows the board-token transformer pipeline: 64 square tokens, a 1024-dimensional embedding, 15 encoder blocks, 32-head attention, and policy plus WDL heads. The current C++ backend is a deterministic visualization model; it does not parse the official LC0 BT4 protobuf as exact trained weights.

9. Keyboard shortcuts

Shortcut	Action
`F`	Flip board
`R`	Reset to the configured start position
`L`	Open Load FEN
`S`	Save current position to `position.fen`
`A`	Enter Setup editor
`E`	Start/stop the native search preview
`F11`	Toggle borderless fullscreen
`Ctrl+Z`	Undo one ply
`Ctrl+Y`	Redo one ply
`Left` / `Right`	Step backward / forward one ply
`Home` / `End`	Jump to the start / end of move history
`Esc`	Deselect, close modal, or leave transient input state

10. Configuration file

`config.ini` controls startup options. Common keys:

```
window.width=1280
window.height=800
window.fullscreen=true
assets.pieces=assets/pieces
board.palette=red
startup.arch=off
default.fen=rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
model.nnue=models/nnue-halfkp-demo.bin
model.lc0_cnn=models/lc0-cnn-112p-10x128-policy4672-wdl3.bin
model.lc0_bt4=models/lc0-bt4-1024x15x32h-visual.bin
clock.enabled=false
```

`board.palette` accepts `red`, `classic`, `neural`, `blue`, or `green`. The default config uses `red`, the warm cream/rose board design. `startup.arch` accepts `nnue`, `cnn`, `bt4`, or `off`.

11. Restrictions and known limitations

- Move input is mouse-only; there is no algebraic or UCI move-entry box.
- PGN export exists in the codebase, but PGN import is not implemented.
- FEN loading expects a complete standard six-field FEN.
- There is no engine opponent yet; play mode is human-vs-human.
- The BT4 `.bin` file validates the visualizer metadata, but the current BT4 panel uses a native deterministic transformer path rather than exact LC0 BT4 trained-weight inference.
- Save FEN writes to `position.fen`; there is no file picker yet.

12. Troubleshooting

- If typed FEN input appears wrong, confirm the active keyboard/input method is plain Latin text before entering the dialog.
- If pieces are missing, check `assets.pieces` in `config.ini`.
- If model panels report missing weights, check the `model.*` paths in `config.ini` and rerun `./scripts/import_models.sh`.